

# Comparing Loops

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Trace each loop: one row per run of the loop body.

## Receipt

```
record Receipt(int[] pricesCents) {  
    int totalCents() {  
        int total = 0;  
        for(int c: pricesCents) {  
            total = total + c;  
        }  
        return total;  
    }  
}  
  
int[] prices = { 350, 125, 499 };  
Receipt lunch = new Receipt(prices);  
int lunchTotal = lunch.totalCents();
```

Q1. Trace lunch.totalCents().

| c | total before | total after |
|---|--------------|-------------|
|   |              |             |
|   |              |             |
|   |              |             |

## ItemList

```
record ItemList(String[] items) {  
    String join(String sep) {  
        String all = "";  
        for(String s: items) {  
            all = all + s + sep;  
        }  
        return all;  
    }  
}  
  
String[] items = { "eggs", "milk", "bread" };  
ItemList groceries = new ItemList(items);  
String list = groceries.join(", ");
```

Q2. Trace groceries.join(", ").

| s | all before | all after |
|---|------------|-----------|
|   |            |           |
|   |            |           |
|   |            |           |

## Stack

```
record Stack(Pict[] picts) {  
    Pict stacked() {  
        Pict pic = ???;  
        for(Pict p: picts) {  
            pic = pic.above(p);  
        }  
        return pic;  
    }  
}  
  
Pict[] stripes = { new Rect(60, 10, "black"),  
                  new Rect(60, 10, "red"),  
                  new Rect(60, 10, "yellow") };  
Stack germany = new Stack(stripes);  
Pict germanFlag = germany.stacked();
```

Q3. pic needs a starting picture that, placed above the first piece, leaves just that piece. What could go where ??? is?

Q4. Trace germany.stacked(). Sketch each picture.

| p | pic before | pic after |
|---|------------|-----------|
|   |            |           |
|   |            |           |
|   |            |           |

## Same loop, three ways

Q5. Fill in what differs between the three methods.

|          | accumulator type | starting value | update |
|----------|------------------|----------------|--------|
| Receipt  |                  |                |        |
| ItemList |                  |                |        |
| Stack    |                  |                |        |

Q6. What does each method return when its array is empty?

totalCents(): \_\_\_\_\_ join(", "): \_\_\_\_\_ stacked(): \_\_\_\_\_

Q7. Change one line of stacked so the pieces line up left to right instead.

## Best so far

Each record gets one more method (... is the method from the front). Every `Pict` has a `width()` method that gives its width in pixels.

```
record Receipt(int[] pricesCents) { ...
    int mostExpensive() {
        int most = pricesCents[0];
        for(int c: pricesCents) {
            if(c > most) {
                most = c;
            }
        }
        return most;
    }
}
int[] prices = { 350, 125, 499 };
Receipt lunch = new Receipt(prices);
```

Q8. Trace `lunch.mostExpensive()`.

| c | true/false | most after |
|---|------------|------------|
|   |            |            |
|   |            |            |
|   |            |            |

```
record ItemList(String[] items) { ...
    String longestItem() {
        String longest = items[0];
        for(String s: items) {
            if(s.length() > longest.length()) {
                longest = s;
            }
        }
        return longest;
    }
}
String[] items = { "eggs", "milk", "bread" };
ItemList groceries = new ItemList(items);
```

Q9. Trace `groceries.longestItem()`.

| s | true/false | longest after |
|---|------------|---------------|
|   |            |               |
|   |            |               |
|   |            |               |

```
record Stack(Pict[] picts) { ...
    Pict widestPict() {
        Pict widest = picts[0];
        for(Pict p: picts) {
            if(p.width() > widest.width()) {
                widest = p;
            }
        }
        return widest;
    }
}
Pict[] parts = { new Circle(10, "gray"),
                 new Circle(15, "gray"),
                 new Circle(20, "gray") };
Stack snowman = new Stack(parts);
```

Q10. Trace `snowman.widestPict()`. Sketch each picture.

| p | true/false | widest after |
|---|------------|--------------|
|   |            |              |
|   |            |              |
|   |            |              |

Q11. In all three traces, the first row's `if` is false. Why?

Q12. All of germany's stripes are 60 wide. Which stripe does `germany.widestPict()` return? \_\_\_\_\_ Change one character of `widestPict()` so it returns the yellow one instead: \_\_\_\_\_

Q13. What happens when each of these runs on an empty array? Can that be avoided?

## More methods

Q14. For each method, name the kind of loop (counter, running-sum, or best-so-far) and give the accumulator's starting value.

| record   | method                                                    | kind of loop | starts at |
|----------|-----------------------------------------------------------|--------------|-----------|
| Receipt  | <code>int countOver(int cents): prices over cents</code>  |              |           |
| ItemList | <code>int totalLetters(): letters in all the items</code> |              |           |
| Stack    | <code>int height(): how tall stacked() is</code>          |              |           |
| Stack    | <code>int width(): how wide stacked() is</code>           |              |           |